Lab 02

Linux Commands and File System

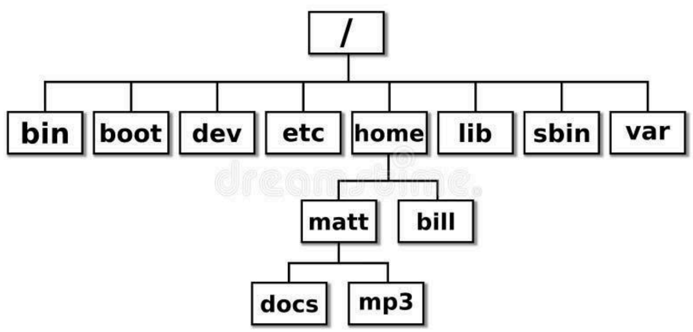
Course: Operating System **Class: AI Section A and B**

Instructor: Bilal Ahmed

Learning Objectives:

- Navigate the file system.
- Basic file manipulation.
- Installing a package.
- Checking Permissions.
- Compile and run the C code.

Basic File System in Linux: Linux follows the Filesystem Hierarchy Standard (FHS) and provides a structured organization for various system files and resources as below shown.



1. **/bin (Binary):**

   Importance: Contains essential binary files and commands crucial for system boot and recovery.
   Example: Key system commands like ls (list files) and cp (copy) are located here. These are fundamental for basic system operation.
2. **/boot:**

   Importance: Holds the kernel, bootloader configuration, and necessary files for system booting.
   Example: The Linux kernel (vmlinuz) and bootloader configurations (like grub.cfg) are stored here, ensuring the system boots successfully.

3. **/dev (Device):**

   Importance: Houses device files representing physical and virtual devices connected to the system.
   Example: /dev/sda represents the first hard drive, /dev/null is a special file for discarding data, and /dev/random provides random data.

4. **/etc :**

   Importance: Contains configuration files and directories for system-wide settings.
   Example: The /etc/network/interfaces file holds network configuration settings, while /etc/apt/sources.list lists package repositories.

5. **/home:**

   Importance: Home directories for regular users, containing personal files and settings.
   Example: If a user "alice" has a home directory /home/alice, her personal files, documents, and user-specific configurations will be stored here.

6. **/lib (Library):**

   Importance: Essential shared libraries and kernel modules required for system functionality.
   Example: Dynamic Link Libraries (DLLs) and shared system libraries are stored here, providing necessary functions for applications.

7. **/sbin (System Binary):**

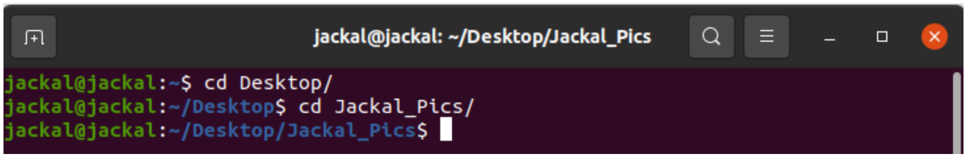
   Importance: Contains system binaries and commands used for system administration and maintenance.
   Example: System administration commands like fdisk (disk partitioning) and ifconfig (network interface configuration) are located here.

8. **/var (Variable):**

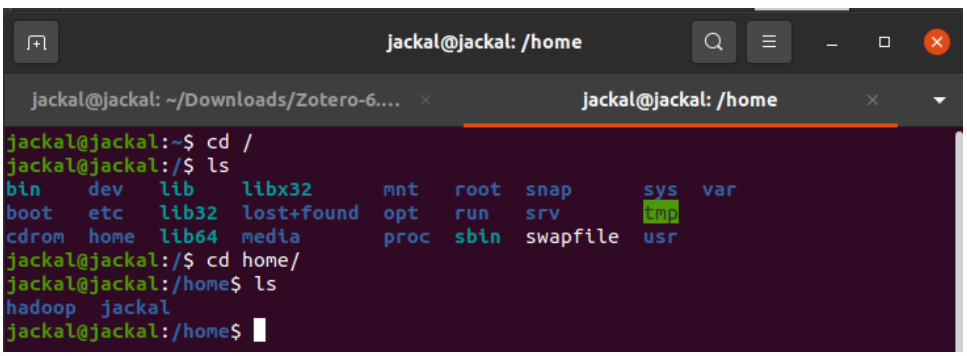
   Importance: Holds variable data files, such as logs, databases, and temporary files that may change during system operation.
   Example: System logs (/var/log/), spool directories (/var/spool/), and temporary files (/var/tmp/) are stored here.

Basic Navigating and file manipulation commands

- **cd -** Changes the current working directory. For example, cd Documents would move to the "Documents" directory.



- **cd .. -** Navigate to previous directory
- **cd/ -** Navigate to root directory.



- **cd or cd~** - Navigate to Home directory
- **mv - Move:** Moves or renames files or directories. For example, mv oldfile.txt newfile.txt renames "oldfile.txt" to "newfile.txt."
- **cp – (Copy):** Copies the file to another folder or content of the file to another file.
- **rm - Remove:** Removes files or directories. Use with caution, as it deletes files without confirmation. For example, rm file.txt deletes "file.txt."
- **cat - Concatenate and Display:** Displays the contents of a file. For example, cat file.txt shows the content of "file.txt."

```
jackal@jackal:~$ cat mac.txt
0C:CB:85:06:AD:5D moto
B8:27:EB:79:47:01 PSX
78:2B:46:87:03:E8 ASIF-PC
e4:5f:01:17:96:31 burger
B8:27:EB:BA:A4:A2 waffle
d8:61:62:b3:a0:21 cpr-j100-0437
d8:61:62:b3:a0:21 Jackal1
e4:70:b8:4e:07:b9 ROSPC
28:24:ff:4d:20:cf Nao2
```

```
ros2@ros2:~$ ls
Desktop    Downloads  Pictures  snap       test
Documents  Music      Public    Templates  Videos
ros2@ros2:~$ rm test
ros2@ros2:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  snap  Templates  Videos
ros2@ros2:~$ touch test
ros2@ros2:~$ mv test new
ros2@ros2:~$ ls
Desktop    Downloads  new       Public  Templates
Documents  Music      Pictures  snap    Videos
ros2@ros2:~$ cat > new
something
^C
ros2@ros2:~$ cp new old
ros2@ros2:~$ ls
Desktop    Downloads  new  Pictures  snap       Videos
Documents  Music      old  Public    Templates
ros2@ros2:~$ cat old
something
ros2@ros2:~$ 
```

- **rmdir:** Removes empty directory.
- **rm -r:** Removes non-empty directory (-r stands for recursive)
- **Cat :** Multipurpose command
  - **cat:** show content of the file
  - **cat >** : edit and write the content in file
  - **cat >>** : edit and append the content in file.

```
ubuntu@Ubuntu:~$ nano some
ubuntu@Ubuntu:~$ cat some
Hi there
ubuntu@Ubuntu:~$ cat hi
cat: hi: No such file or directory
ubuntu@Ubuntu:~$ nano hi
ubuntu@Ubuntu:~$ cat hi some
jkl
Hi there
ubuntu@Ubuntu:~$ 
```

```
ubuntu@Ubuntu:~$ cat > hi
Hello dear
ubuntu@Ubuntu:~$ ^C
ubuntu@Ubuntu:~$ cat hi
Hello dear
ubuntu@Ubuntu:~$ cat > hi
Yes^C
ubuntu@Ubuntu:~$ cat hi
Yesubuntu@Ubuntu:~$
```

```
ubuntu@Ubuntu:~$ cat > hi
Yes^C
ubuntu@Ubuntu:~$ cat hi
Yesubuntu@Ubuntu:~$ cat >> hi
No^C
ubuntu@Ubuntu:~$ cat hi
YesNoubuntu@Ubuntu:~$ cat >> hi

Hello
^C
ubuntu@Ubuntu:~$ cat hi
YesNo
Hello
ubuntu@Ubuntu:~$
```

- **nano or vi - Text Editors:** Opens a text editor. For example, nano filename or vi filename allows you to edit the contents of a file.

```
jackal@jackal: ~
  GNU nano 4.8                          mac.txt
0C:CB:85:06:AD:5D moto
B8:27:EB:79:47:01 PSX
78:2B:46:87:03:E8 ASIF-PC
e4:5f:01:17:96:31 burger
B8:27:EB:BA:A4:A2 waffle
d8:61:62:b3:a0:21 cpr-j100-0437
d8:61:62:b3:a0:21 Jackal1
e4:70:b8:4e:07:b9 ROSPC
28:24:ff:4d:20:cf Nao2
```

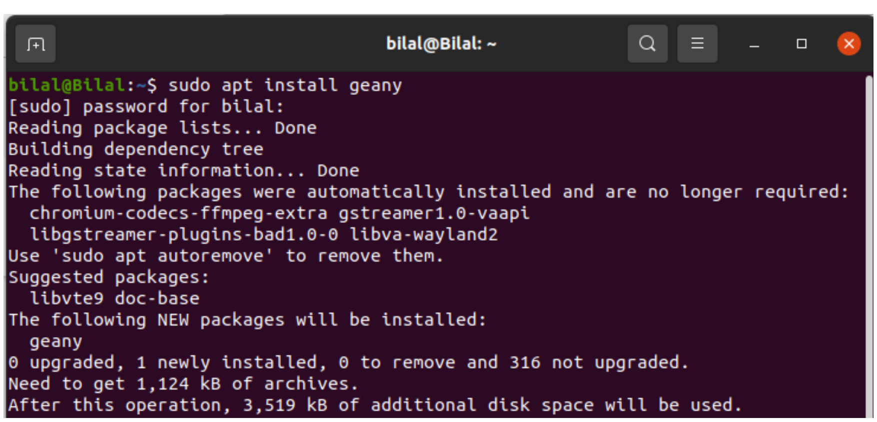
- **man - Manual Pages:** Displays the manual or help pages for a command. For example, man ls provides information about the ls command.

```
jackal@jackal: ~/Downloads/Zotero-6....        jackal@jackal: ~
LS(1)                            User Commands                           LS(1)

NAME
       ls - list directory contents

SYNOPSIS
       ls [OPTION]... [FILE]...

DESCRIPTION
       List  information  about  the FILEs (the current directory by default).
       Sort entries alphabetically if none of -cftuvSUX nor --sort  is  speci-
       fied.

       Mandatory  arguments  to  long  options are mandatory for short options
       too.

       -a, --all
              do not ignore entries starting with .

       -A, --almost-all
              do not list implied . and ..

       --author
 Manual page ls(1) line 1 (press h for help or q to quit)
```

- **echo - Print to Console:** *Description:* Prints text to the console. For example, echo "Hello, Linux!" displays "Hello, Linux!" on the screen.

Installing a package in Linux: Installing a package or a software can be done using both GUI and command line. Among these two, command line is a better choice for which linux is famous and widely being used due to its powerful package installation and dependency handling. In Debian-based distributions such as Ubuntu, Advance Packaging Tool (APT) is widely used. Let's install a simple editor named Geany using apt in linux. It is always a good practice to update the system using update command as *sudo apt update.*

Package installation command: *sudo apt install geany*



```
bilal@Bilal: ~
bilal@Bilal:~$ sudo apt install geany
[sudo] password for bilal:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  chromium-codecs-ffmpeg-extra gstreamer1.0-vaapi
  libgstreamer-plugins-bad1.0-0 libva-wayland2
Use 'sudo apt autoremove' to remove them.
Suggested packages:
  libvte9 doc-base
The following NEW packages will be installed:
  geany
0 upgraded, 1 newly installed, 0 to remove and 316 not upgraded.
Need to get 1,124 kB of archives.
After this operation, 3,519 kB of additional disk space will be used.
```

Output: Generally, if the package size is small, it will install it directly, otherwise it will ask to confirm the installation by asking y (yes) or n (no).

File Permissions in Linux

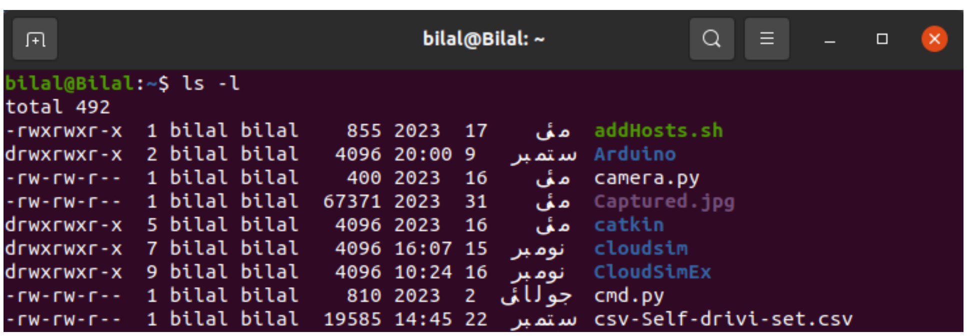
Owner types in Linux: Linux has three types of owners; **user, group,** and **others.**

User: Default owner and creator of the file.
Group: Collection of users assigned a particular set of permissions, such as editors and viewers.
Others: Apart from user and group.

Permission types in Linux: Linux has 3 types of permissions; **Read, Write** and **Execute.** Symbolic Representation of file permission to groups can be seen below.

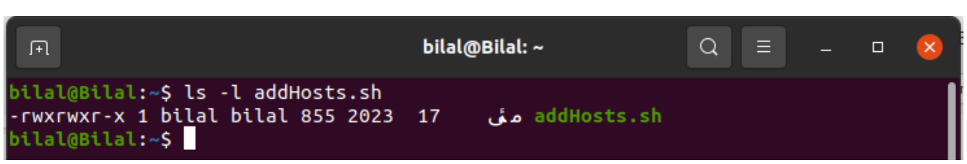
User			Group			Others		
read	write	execute	read	write	execute	read	write	execute

Checking file permissions with respect to the owner using ***ls -l*** command as below.



Explanation: (-) Shows file and d stands for directory. The first set of permissions is for User, second for group and third for others. For example, addHosts.sh is file which has r (read) w (write) and x (execute) permission to User and Group, and only executable permission to others group.

For Specific File: To check permissions of specific file, execute ls -l <filename> as below shown.



Compiling C code:

Writing Code: Below is simple C code. Create file using nano and add name this as hello.c

```
#include <stdio.h>

int main() {
    // This is a simple C program
    printf("Hello, World!\n");
    return 0;
}
```

Compile: Compile it using gcc command and produce its executable file.

Compile command: gcc hello.c -o hello

Execution command: bash hello.c

Exercise.

Task 1: Compare file hierarchy system of Windows, Linux and MacOS.

Task 2: Ubuntu supports other package manager tools except its default **apt,** some of them are listed below, install , use, compare and discuss them with example.

- Aptitude
- Snap
- Dpkg
- Synaptic

Task 3: Compiling and running a C Program.

- Write a C Program which should print your Name, Department and batch.
- Check its permissions.
- Compile the program and create its executable.
- Execute the compiled C Program.

Real Life Importance of this lab

System Administration:

Scenario: Imagine you are a system administrator responsible for managing a server infrastructure. You need to navigate the file system to troubleshoot issues, check log files in /var, and update system configurations in /etc. Understanding basic commands like ls, cd, and file permissions (chmod and chown) is essential for efficient system maintenance.

Software Development:

Scenario: As a software developer, you are working on a Linux-based project. You need to compile and run C code, manage different versions of your software, and understand file permissions for source code repositories. Knowing how to use gcc for compilation and managing file permissions in the development environment is critical.

Package Management:

Scenario: Suppose you are developing a web application that relies on specific software packages. In Lab 02, you learned how to use package management tools (apt) to install necessary software. This skill is crucial for ensuring that all dependencies are met for your application to run smoothly.

User Account Management:

Scenario: In a corporate setting, you are responsible for creating and managing user accounts on a Linux server. Understanding how to add users (adduser), check user IDs (id), and change file ownership (chown) is vital for maintaining a secure and organized user environment.

Security Auditing:

Scenario: As part of a security audit, you need to review and adjust file permissions on critical system files. Knowing how to use chmod to modify permissions and ls -l to check current permissions helps in ensuring that sensitive information is properly secured.

Server Boot and Recovery:

Scenario: In the event of a system failure, you need to troubleshoot and recover the system. Understanding the contents of the /boot directory and using recovery tools located in /bin and /sbin are essential for bringing the system back online.

Configuration Management:

Scenario: You are working in a DevOps role, and automation is key. Understanding file system structure (/etc for configurations), package management, and user account management is crucial for creating and managing automated deployment scripts and configuration files.